

Importante: Los ejercicios deben entregarse a través de web (**Domjudge y Blackboard**). Cada ejercicio deberá ir en un fichero con nombre:

ColaPrioridad.h

Supermercado.h

ListaContigua.h

ListaContigua.cpp

mainSupermercado.cpp

ColaPrioridad.cpp

Supermercado.cpp

Los cinco primeros ficheros podrán ser descargados a través de **Blackboard** y no deberán modificados por el alumno. Sin embargo, *ColaPrioridad.cpp* y *Supermercado.cpp*, deberán ser desarrollados completamente por el alumno.

La fecha de entrega: consultar la página de la actividad en blackboard

Supermercado con prioridad (3 puntos): Se quiere gestionar el funcionamiento de las cajas de un supermercado para poder saber que clientes se encuentran en cada una de las cajas y en caso de cerrar una de ellas recolocar a todos los clientes en las restantes. Para este propósito se deberán implementar dos clases: la clase *ColaPrioridad* que permite almacenar los clientes que se encuentran esperando en una caja y que se van atendiendo según el orden de llegada y la clase *Supermercado* que permite gestionar varias cajas al mismo tiempo. La clase *ColaPrioridad* se implementará mediante un montículo para asegurar que todos los clientes sean servidos según su orden de llegada. Esta clase se compone de los siguientes atributos:

- **ListaContigua vector.** Lista contigua sobre la que se implementará la cola de prioridad (Montículo).

Y será necesario implementar los siguientes métodos:

- **void encolar(int num).** Recibe un número que representará el orden de llegada del cliente al supermercado y lo colocará al final de la cola. Sin embargo al ser una cola de prioridad este elemento irá escalando hasta la posición que le corresponda dentro del Montículo. Es decir, se irá intercambiando con su padre hasta que sea mayor que su padre o hasta que lleguemos a la raíz
- **int desencolar().** Devuelve el número que representa el orden de llegada del cliente que se encuentra en la primera posición del montículo. Al ser una cola de prioridad se deberán seguir los siguientes pasos:
 - El primer elemento será sustituido por el último elemento.

- o El último elemento de la cola será eliminado
- o Se deberá reestructurar el montículo usando el método reestructurar.
- **bool estaVacia()**. Indica si la cola se encuentra vacía.
- **void reestructurar()**. Después de haber desencholado un elemento, el montículo puede no ser un montículo así que esta función se encarga de reestructurar el montículo para que sea otra vez un montículo.

Por otro lado la clase Supermercado tiene los siguientes atributos:

- **ColaPrioridad *cajas**. Array de punteros a objetos de tipo ColaPrioridad que representan cada una de las cajas..
- **int n_cajas**. Número de cajas que hay en el supermercado

Y será necesario implementar los siguientes métodos::

- **Supermercado(int n)**. Constructor que se encarga de reservar memoria para las n cajas que tiene el supermercado e inicializa el atributo n_cajas.
- **void nuevoUsuario(int n,int id)**. Encola el usuario con el id indicado en la caja que se encuentra en la posición n del array de cajas.
- **void cerrarCaja(int n)**. Esta función simula el cierre de la caja n y el reparto de los usuarios en las cajas restantes. Para ello será necesario desencolar todos los usuarios que se encuentran en la caja n en el orden de llegada, e ir encolándolos en las cajas restantes que no estén vacías. Debido a que los usuarios se deberán repartir de forma equitativa en las cajas restantes, iremos recorriendo las cajas por orden de 0 a N-1 e introduciremos un único usuario por cada caja que no esté vacía. Cuando hayamos introducido un usuario en cada una de las cajas volveremos a empezar desde la caja 0 hasta que no queden más usuarios. Debido al uso de colas de prioridad, los clientes que hayan llegado antes se ordenaran de forma correcta en cada una de las cajas restantes según su orden de llegada.
- **int atenderUsuario(int n)**; Atiende al usuario que se encuentra en la caja n y por tanto lo desencola de la cola que representa dicha caja. Esta función devolverá el id del usuario atendido.
- **bool cajaVacia(int n)**. Indica si la caja n tiene o no tiene usuarios esperando.

Utilizar el fichero mainSupermercado.cpp proporcionado a través de Blackboard para realizar las pruebas necesarias y para enviar al corrector automático. Este programa simula el funcionamiento de un supermercado con las siguientes opciones:

- N: Crea un objeto de tipo supermercado que permitirá controlar varias cajas.

Algoritmos y estructuras de datos

- U: Encola un usuario en una de las cajas.
- C: Simula el cierre de una de las cajas repartiendo los usuarios entre las cajas restantes
- A: Atiende al primer usuario que se encuentre en la caja indicada
- S: Termina.